VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

Faculty of Computer Science and Engineering

Course CO3021 — Database Management Systems



DATABASE MANAGEMENT SYSTEMS PROJECT REPORT

Clinical Case-Study: Low Back Pain Diagnosis & Imaging Pipeline

---

Workload-Driven Evaluation of Microsoft SQL Server 2025:

Storage Architectures, Relational Ingestion, and AI Feature Extraction Pipeline (Role 1)

Applied to 48,345 Lumbar Spine MRI Scans and Radiologist Clinical Reports

---

Group 02

Course Instructor: PhD. Phan Trong Nhan

Project Repository: https://github.com/qmarcusv/DBMS_TA_Group2

Ho Chi Minh City, October 2026

Contents

1 Clinical MRI Data Pipeline, Storage Architecture, and Ingestion Engineering (Role 1) 1

1.1 Clinical Problem Context and Dataset Overview . . . 1
1.2 The Three Canonical Clinical Queries Contract . . . 1
1.3 Theoretical Foundations of Database Storage . . . 2
1.4 Evaluation of MSSQL Binary and Unstructured Storage Technologies . . . 2
1.4.1 Architectural Comparison of Storage Approaches . . . 2
1.4.2 Architectural Rationale for Relative Path Selection . . . 2
1.5 Schema Architecture and Relational Integrity Constraints . . . 2
1.6 Comparative Evaluation of Data Ingestion Pipelines . . . 3
1.7 Query 3 Implementation: Deep Learning Feature Extraction on CPU . . . 4
1.7.1 Pipeline Architecture and CPU Parallelization . . . 4
1.7.2 Mathematical Similarity Metric in T-SQL . . . 4
1.7.3 Relational DBMS vs. Dedicated Vector Database Tradeoffs . . . 4
1.8 Analysis Across All Three Canonical Queries . . . 4

2 Physical Storage and Index Design (Role 2: Storage & Indexing) 5

2.1 Physical Storage and B-Tree Depth Analysis . . . 5
2.1.1 Baseline Database State . . . 5
2.1.2 B-Tree Depth Observations . . . 5
2.1.3 Physical Bottleneck in the Baseline State . . . 5
2.2 Index Design and Implementation . . . 5
2.2.1 Covering B-Tree Indexes for Q1 and Q2 . . . 5
2.2.2 Filtered Index for T2 Sagittal Sequences (Q2) . . . 5
2.2.3 Non-Clustered Columnstore Index (NCCI) for Q3 . . . 6
2.3 Execution Plan Analysis . . . 6
2.3.1 Q1 and Q2: Before and After Covering B-Tree Indexing . . . 6
2.3.2 Q3: Rowstore Parallel Scan versus Columnstore Batch Mode . . . 7
2.4 Benchmark Evaluation: I/O and CPU Measurements . . . 8
2.4.1 Discussion of Results . . . 8
2.4.2 Chapter Summary . . . 9

3 Query Optimization and Query Processing Analysis 9

3.1 Introduction . . . 9
3.2 Q1: Patient and MRI Scan Retrieval . . . 9
3.2.1 Query Characteristics . . . 9
3.2.2 Execution Plan and Join Selection . . . 9
3.2.3 Cardinality Estimation . . . 10
3.2.4 Statistics for `MRI_Scans.PatientID` . . . 10
3.2.5 Runtime and I/O Observations . . . 11
3.3 Q3: Nearest-Neighbor Feature Search . . . 11
3.3.1 Query Characteristics . . . 11
3.3.2 Execution Plan and Join Algorithm . . . 12

3.3.3 Parallel Execution . . . 12
3.3.4 Statistics Used for Cardinality Estimation . . . 13
3.3.5 Compute Scalar and Vector Distance Calculation . . . 14
3.3.6 Top-N Sort and Result Selection . . . 14
3.3.7 Reference-Vector Lookup . . . 15
3.3.8 Runtime, I/O, and Memory Observations . . . 15
3.4 Cross-Query Analysis . . . 16
3.4.1 Join Algorithm Comparison . . . 16
3.4.2 Cardinality Estimation Comparison . . . 16
3.4.3 Access Path and Execution Behaviour Comparison . . . 17
3.5 Findings and Discussion . . . 17

Chapter 1: Clinical MRI Data Pipeline, Storage Architecture, and Ingestion Engineering (Role 1)

1.1 Clinical Problem Context and Dataset Overview

Modern Picture Archiving and Communication Systems (PACS) and Radiology Information Systems (RIS) manage heterogeneous medical data comprising structured diagnostic records and voluminous medical imagery. This project addresses the clinical workflow of diagnosing Low Back Pain (LBP) and Lumbar Spinal Stenosis (LSS) using the Mendeley Clinical Lumbar Spine MRI dataset:

- **Radiologist Clinical Notes:** 575 clinical narrative reports detailing lumbar pathology including disc herniation, annular tears, thecal sac compression, and neural foramen stenosis.
- **Multimodal MRI Scans:** 48,345 DICOM slices (`.ima`) distributed across 515 patients in 4-to-5 level nested directory structures covering sagittal (T1, T2) and axial sequences.

1.2 The Three Canonical Clinical Queries Contract

Following the instructor's updated evaluation directives, all sub-roles evaluate and interpret DBMS behavior across the exact three canonical clinical queries engineered by Role 1:

Query 1 (Q1) — Point Lookup and One-to-Many Clinical Join

Retrieves the complete narrative diagnostic report alongside all associated MRI scan paths for a designated patient:

```
SELECT p.PatientID, p.PatientName, p.ClinicalNotes,
       s.ScanID, s.SeriesDescription, s.ImagePath
FROM Patients p
INNER JOIN MRI_Scans s ON p.PatientID = s.PatientID
WHERE p.PatientID = '0001';
```

Query 2 (Q2) — Sequence-Selective Diagnostic Filtering

Retrieves imaging slices restricted to sagittal T2-weighted pulse sequences (`T2_TSE_SAG`) frequently utilized for spinal cord and disc herniation assessment:

```
SELECT p.PatientID, p.PatientName, p.ClinicalNotes,
       s.ScanID, s.SeriesDescription, s.ImagePath
FROM Patients p
INNER JOIN MRI_Scans s ON p.PatientID = s.PatientID
WHERE p.PatientID = '0001' AND s.SeriesDescription LIKE '%T2%SAG%';
```

Query 3 (Q3) — Content-Based Image Retrieval via Vector Similarity

Calculates mathematical proximity (Euclidean distance and cosine similarity) between an index query slice ($ScanID = 1$) and all other scans in the database using high-dimensional feature embeddings:

```
DECLARE @TargetScanID INT = 1;
DECLARE @T1 FLOAT, @T2 FLOAT, @T3 FLOAT, @T4 FLOAT;
SELECT @T1 = V_Dim1, @T2 = V_Dim2, @T3 = V_Dim3, @T4 = V_Dim4
FROM Image_Features WHERE ScanID = @TargetScanID;

SELECT TOP 3 s.ScanID, s.PatientID, s.SeriesDescription, s.ImagePath,
    SQRT(SQUARE(f.V_Dim1 - @T1) + SQUARE(f.V_Dim2 - @T2) +
         SQUARE(f.V_Dim3 - @T3) + SQUARE(f.V_Dim4 - @T4)) AS EuclideanDistance
FROM Image_Features f
INNER JOIN MRI_Scans s ON f.ScanID = s.ScanID
WHERE f.ScanID <> @TargetScanID
ORDER BY EuclideanDistance ASC;
```

1.3 Theoretical Foundations of Database Storage

In relational database theory, storage subsystems organize records into fixed-size physical blocks or pages. In Microsoft SQL Server, the fundamental unit of I/O is the **8KB Data Page** (8,192 bytes), of which 8,060 bytes accommodate user data after subtracting page headers and slot arrays. When storing binary large objects (BLOBs) exceeding 8KB, conventional row architectures fragment records across off-row text/image tree allocations. This architectural reality presents critical performance tradeoffs when designing clinical multimedia databases.

1.4 Evaluation of MSSQL Binary and Unstructured Storage Technologies

To handle 13GB of DICOM MRI scans, Microsoft SQL Server provides four distinct architectural mechanisms:

1.4.1 Architectural Comparison of Storage Approaches

1. **In-Row and Out-of-Row `VARBINARY(MAX)`:** Binary bytes reside directly within the `.mdf` database file. Slices larger than 8KB migrate to dedicated LOB pages. *Drawback:* Loading medical images into memory causes severe **Buffer Pool thrashing**, displacing critical index and table pages from RAM cache and inflating backup archives to multi-gigabyte sizes.
2. **FILESTREAM Storage:** Combines the SQL Server Database Engine with the Windows NTFS/ReFS file system. Unstructured files reside in operating system directories while remaining under SQL Server transactional control. *Mechanism:* Data transfers occur via native Win32 streaming APIs, **completely bypassing the SQL Server Buffer Pool** and preserving cache memory for relational query processing.
3. **FileTable Technology:** Built upon FILESTREAM, FileTable integrates a Windows Win32 namespace hierarchy. Operating system users and external PACS systems can drag, drop, and edit `.ima` files via standard Windows Network Shares (SMB), while SQL Server automatically reflects changes within relational tables.
4. **Relative File Path Reference (`NVARCHAR(500)`):** The database stores only relative filesystem paths pointing to operating system directories. The physical binary assets remain completely decoupled from the relational engine.

Table 1.1: Comprehensive Comparison of Large Binary (BLOB) Storage Architectures in MSSQL

Evaluation Metric	VARBINARY(MAX)	FILESTREAM	FileTable	Relative Path (Selected)
Physical Location	Database `.mdf` file	NTFS/ReFS directories	NTFS via SMB share	External file directory
Buffer Pool Cache	Heavy RAM pollution	Bypasses Buffer Pool	Bypasses Buffer Pool	Zero Buffer Pool footprint
ACID Transactionality	Full native logging	Full Win32 transactional	Full Win32 transactional	Application-level control
Backup Archive Size	13GB+ database backup	Included in backup stream	Included in backup stream	Compact 53MB metadata backup
AI Framework Access	Requires SQL extraction	Win32 API streaming	Direct Windows file path	Native Python `pydicom` I/O

1.4.2 Architectural Rationale for Relative Path Selection

For this research workload, the **Relative File Path** architecture was selected. Decoupling the 13GB image binaries from the database reduced the database footprint from 13GB to 53MB. This prevents Buffer Pool contention during Indexing and Query Optimization benchmarks while enabling native, multi-threaded PyTorch feature extraction on disk without database roundtrip latency.

1.5 Schema Architecture and Relational Integrity Constraints

Rather than allowing SSMS Import Wizards to infer loose schemas defaulting to `NVARCHAR(MAX)`, strict DDL constraints were implemented to optimize data page packing and enforce relational integrity:

```
CREATE TABLE Patients (
    PatientID VARCHAR(20) NOT NULL,
    PatientName NVARCHAR(100) NULL,
    ClinicalNotes NVARCHAR(MAX) NULL,
```

```
    DiagnosisStatus VARCHAR(20) DEFAULT 'Confirmed',
    CreatedAt DATETIME DEFAULT GETDATE(),
    CONSTRAINT PK_Patients PRIMARY KEY CLUSTERED (PatientID)
);

CREATE TABLE MRI_Scans (
    ScanID INT IDENTITY(1,1) NOT NULL,
    PatientID VARCHAR(20) NOT NULL,
    StudyInstanceUID VARCHAR(150) NULL,
    SeriesDescription NVARCHAR(255) NOT NULL,
    ImagePath NVARCHAR(500) NOT NULL,
    AcquisitionDate DATETIME DEFAULT GETDATE(),
    CONSTRAINT CK_Series_Valid CHECK (SeriesDescription NOT LIKE '%INVALID%'),
    CONSTRAINT PK_MRI_Scans PRIMARY KEY CLUSTERED (ScanID),
    CONSTRAINT FK_MRI_Scans_Patients FOREIGN KEY (PatientID)
        REFERENCES Patients(PatientID) ON DELETE CASCADE ON UPDATE CASCADE
);

CREATE TABLE Image_Features (
    FeatureID INT IDENTITY(1,1) NOT NULL,
    ScanID INT NOT NULL UNIQUE,
    FeatureVector NVARCHAR(MAX) NOT NULL,
    V_Dim1 FLOAT NULL, V_Dim2 FLOAT NULL, V_Dim3 FLOAT NULL, V_Dim4 FLOAT NULL,
    CONSTRAINT PK_Image_Features PRIMARY KEY CLUSTERED (FeatureID),
    CONSTRAINT FK_Features_Scans FOREIGN KEY (ScanID) REFERENCES MRI_Scans(ScanID) ON DELETE
    CASCADE
);
```

Relational Integrity and Constraint Highlights

- **Compact Primary Keys:** `PatientID VARCHAR(20)` ensures high page density within Clustered B-Tree leaf pages compared to unrestricted unicode datatypes.
- **Transaction Test Constraint:** The check constraint `CK_Series_Valid` deliberately rejects records containing `'INVALID'` strings, establishing the deterministic error boundary required by Role 4 to verify atomic transaction rollback.

1.6 Comparative Evaluation of Data Ingestion Pipelines

Bulk data loading was evaluated across four industry standard ingestion methodologies:

Table 1.2: Comparative Benchmark of Data Ingestion Methodologies into SQL Server

Ingestion Method	Mechanism	Logging Mode	Ingestion Speed	Automation Support
SSMS Flat File Wizard	Interactive GUI	Fully Logged	15–20 seconds	Manual interaction
T-SQL BULK INSERT	In-process Server Worker	Minimally Logged (`TABLOCK`)	1.2 seconds	Fully scriptable
Command-Line bcp	External client utility	Minimally Logged (`-h TABLOCK`)	1.1 seconds	CLI and batch files
Python pyodbc	Client cursor array	Fully Logged	8–10 seconds	Integrated AI pipeline

Identity Column Mapping Resolution via View Abstraction

During direct `BULK INSERT` execution into tables containing an auto-incrementing identity column (`ScanID INT IDENTITY`), SQL Server raises error `Msg 7301` due to column mapping mismatches between the 5-column CSV and 6-column physical schema. This was resolved without complex XML format files by creating an updatable relational view:

```
CREATE VIEW dbo.vw_MRI_Scans_Import AS
SELECT PatientID, StudyInstanceUID, SeriesDescription, ImagePath, AcquisitionDate
FROM dbo.MRI_Scans;
```

```
BULK INSERT dbo.vw_MRI_Scans_Import FROM '...clean_mri_scans.csv'
WITH (FIRSTROW = 2, FORMAT = 'CSV', FIELDQUOTE = '"', FIELDTERMINATOR = ',',
      ROWTERMINATOR = '0x0a', CODEPAGE = '65001', TABLOCK);
DROP VIEW dbo.vw_MRI_Scans_Import;
```

1.7 Query 3 Implementation: Deep Learning Feature Extraction on CPU

To support Query 3 (Content-Based Medical Image Retrieval), an end-to-end AI feature extraction pipeline was engineered using PyTorch and `pydicom`:

1.7.1 Pipeline Architecture and CPU Parallelization

1. **DICOM Matrix Ingestion:** Medical slices (`.ima`) are parsed via `pydicom.dcmread`, normalized from 12-bit dynamic range ($[0, 4095]$) to $[0, 255]$ uint8 arrays, and resized to $224 \times 224 \times 3$.
2. **Pretrained ResNet-18 Inference:** Slices are processed in minibatches of 64 through an ImageNet-pretrained ResNet-18 model. The final fully connected classification layer is truncated, extracting 512-dimensional feature embeddings from the penultimate `AdaptiveAvgPool2d` layer.
3. **Vector Normalization and Database Persistence:** Each 512-dimensional embedding $\mathbf{v}$ is normalized via Euclidean norm ($\|\mathbf{v}\|_2 = 1$). The leading dimensions are persisted in float columns (`V_Dim1..V_Dim4`) alongside full JSON vector strings.

Throughput Performance on 20-Core CPU

By leveraging batch inference across 20 CPU threads, the pipeline achieved an average throughput of **20.2 images/second**, extracting all 48,345 slices in **39.9 minutes** without requiring specialized GPU hardware.

1.7.2 Mathematical Similarity Metric in T-SQL

In Query 3, similarity is evaluated using Euclidean Distance:

$$d(\mathbf{u}, \mathbf{v}) = \sqrt{\sum_{k=1}^{4} (u_k - v_k)^2} \tag{1.1}$$

Because feature vectors are L_2-normalized, Euclidean distance relates monotonically to Cosine Similarity ($\|\mathbf{u} - \mathbf{v}\|_2^2 = 2 - 2\cos(\theta)$). Executing Query 3 over all 48,345 scans yielded the top-3 similar slices with sub-100ms execution latency (98ms elapsed time).

1.7.3 Relational DBMS vs. Dedicated Vector Database Tradeoffs

While Microsoft SQL Server executes vector distance calculations cleanly via arithmetic operators (`Compute Scalar`) and memory-bounded sorts (`Top N Sort`), it fundamentally lacks Approximate Nearest Neighbor (ANN) indexing structures such as HNSW (Hierarchical Navigable Small World) or IVF (Inverted File Index). Consequently, vector search in standard RDBMS remains an $\mathcal{O}(N)$ brute-force table scan. In large-scale clinical production, dedicated vector stores (e.g., Milvus, Qdrant) or extensions (`pgvector`) are recommended for billion-scale embeddings.

1.8 Analysis Across All Three Canonical Queries

The engineered data and storage architecture empowers all three clinical query contracts:

- **Impact on Query 1 (Point Lookup):** Returning text paths rather than binary BLOBs prevents network saturation and eliminates Buffer Pool contention.
- **Impact on Query 2 (Sequence Filtering):** Strict `NVARCHAR(255)` sequence typing allows the Query Optimizer to leverage B-Tree index seeks on pulse sequence descriptors without reading external storage.
- **Impact on Query 3 (Similarity Search):** Native float representation enables direct relational calculation of Euclidean distances, seamlessly integrating machine learning embeddings into SQL workflows.

Chapter 2: Physical Storage and Index Design (Role 2: Storage & Indexing)

This chapter presents the work of Role 2 on the SpineMRI_DBMS database. It analyses the physical storage structure and B-Tree depth of the baseline schema, describes the design of three index types (Covering B-Tree, Filtered Index, and Non-Clustered Columnstore Index), analyses the actual execution plans captured in SSMS, and summarises the STATISTICS IO and STATISTICS TIME measurements for queries Q1, Q2, and Q3.

2.1 Physical Storage and B-Tree Depth Analysis

2.1.1 Baseline Database State

The SpineMRI_DBMS database was restored from a 53.6 MB backup file. In this state, the only indexes are the default clustered indexes created on the primary keys. The physical statistics, obtained from the DMV sys.dm_db_index_physical_stats(..., 'DETAILED'), are summarised in Table 2.1.

Table 2.1: Physical storage structure of the tables in the baseline state

Table	Index type	Key	B-Tree depth	Leaf pages	Size	Rows
Patients	Clustered	PatientID	2	8	0.06 MB	575
MRI_Scans	Clustered	ScanID	3	1,876	14.66 MB	48,345
Image_Features	Clustered	ScanID	3	3,900	30.47 MB	48,345

2.1.2 B-Tree Depth Observations

- **Patients table (depth = 2):** a single root page (Level 1) points directly to 8 leaf pages (Level 0). Any lookup by PatientID therefore costs exactly 2 logical page reads (logical reads = 2).
- **MRI_Scans and Image_Features tables (depth = 3):** each consists of a root page (Level 2), intermediate pages (Level 1), and a leaf level (Level 0) holding all 48,345 rows.

2.1.3 Physical Bottleneck in the Baseline State

SQL Server does **not** automatically create an index on foreign key columns such as MRI_Scans.PatientID. Consequently, when Patients is joined to MRI_Scans with the predicate PatientID = '0001', the optimizer is forced to scan all 1,876 leaf pages plus the IAM and non-leaf pages of MRI_Scans, a total of **1,880 logical reads**, merely to retrieve a few dozen matching rows.

2.2 Index Design and Implementation

2.2.1 Covering B-Tree Indexes for Q1 and Q2

Listing 2.1: Two covering B-Tree indexes on MRI_Scans

```
-- 1. Single-key index on SeriesDescription: lighter than the base table for series scans
CREATE NONCLUSTERED INDEX IX_MRIScans_SeriesDescription
ON dbo.MRI_Scans (SeriesDescription ASC)
INCLUDE (PatientID, ImagePath);

-- 2. Composite covering index tuned directly for Q1 and Q2
CREATE NONCLUSTERED INDEX IX_MRIScans_PatientID_Covering
ON dbo.MRI_Scans (PatientID ASC, SeriesDescription ASC)
INCLUDE (ImagePath);
```

Design rationale. Placing PatientID as the leading key allows SQL Server to perform an *Index Seek* that jumps directly to the range holding the 65 slices of patient '0001'. The second key column, SeriesDescription, keeps the sequence names pre-sorted in lexicographic order within each patient. The INCLUDE (ImagePath) clause attaches the image file path to the leaf level without enlarging the upper navigation nodes, so the query obtains all required data from the index itself (**0 Key Lookups**).

2.2.2 Filtered Index for T2 Sagittal Sequences (Q2)

Listing 2.2: Filtered Index for T2 Sagittal sequences

```
CREATE NONCLUSTERED INDEX IX_MRIScans_T2Sag_Filtered
ON dbo.MRI_Scans (PatientID ASC, SeriesDescription ASC)
INCLUDE (ImagePath)
WHERE SeriesDescription IN (
    'I_T2_TSE_SAG_P2_384_0002', 'POSDISP_[2]_T2_TSE_SAG_384_5001',
    'POSDISP_[2]_T2_TSE_SAG_384_5002', 'POSDISP_[3]_T2_TSE_SAG_384_5001',
    'POSDISP_[4]_T2_TSE_SAG_384_5001', 'POSDISP_[6]_T2_TIRM_SAG_P2_5002',
    'POSDISP_[7]_T2_TIRM_SAG_P2_5003', 'T2_CISS3D_SAG_ISO_0006',
    'T2_TIRM_SAG_BOX_0008', 'T2_TIRM_SAG_P2_0004', 'T2_TIRM_SAG_P2_0005',
    'T2_TIRM_SAG_P2_0006', 'T2_TIRM_SAG_P2_0007', 'T2_TIRM_SAG_P2_0008',
    'T2_TIRM_SAG_P2_0009', 'T2_TIRM_SAG_P2_0010', 'T2_TSE_SAG_256_0006',
    'T2_TSE_SAG_256_0007', 'T2_TSE_SAG_320_0002', 'T2_TSE_SAG_320_0003',
    'T2_TSE_SAG_380_0006', 'T2_TSE_SAG_380_0008', 'T2_TSE_SAG_384_0002',
    'T2_TSE_SAG_384_0003', 'T2_TSE_SAG_384_0004', 'T2_TSE_SAG_384_0005',
    'T2_TSE_SAG_384_0006', 'T2_TSE_SAG_384_0007', 'T2_TSE_SAG_384_0008',
    'T2_TSE_SAG_384_0009', 'T2_TSE_SAG_448_0006', 'T2_TSE_SAG_448_0013'
);
```

Key technical point. In Microsoft SQL Server, the filtered index predicate parser does *not* allow the `LIKE` operator with a leading wildcard. The statement `WHERE SeriesDescription LIKE '%T2%SAG%'` fails with `Msg 10735: Incorrect WHERE clause for filtered index`. We resolved this by profiling the actual value domain of the column in `MRI_Scans`, extracting exactly **32 distinct T2 Sagittal sequence codes**, and constructing a valid `WHERE SeriesDescription IN (...)` clause.

2.2.3 Non-Clustered Columnstore Index (NCCI) for Q3

Listing 2.3: Non-Clustered Columnstore Index on `Image_Features`

```
CREATE NONCLUSTERED COLUMNSTORE INDEX NCCI_ImageFeatures_Dims
ON dbo.Image_Features (ScanID, V_Dim1, V_Dim2, V_Dim3, V_Dim4);
```

Design rationale. Query Q3 computes the Euclidean distance over all 48,345 rows of `Image_Features`. Under rowstore storage, each 8 KB page must also hold the long JSON vector string column (`NVARCHAR(MAX)`), so the table occupies 3,909 disk pages. The columnstore index separates the four floating-point columns (`V_Dim1..V_Dim4`) and `ScanID` into compressed column segments, and enables **Batch Mode** execution, which processes roughly 900 rows per batch using vectorised CPU operations instead of row-by-row Row Mode processing.

2.3 Execution Plan Analysis

2.3.1 Q1 and Q2: Before and After Covering B-Tree Indexing

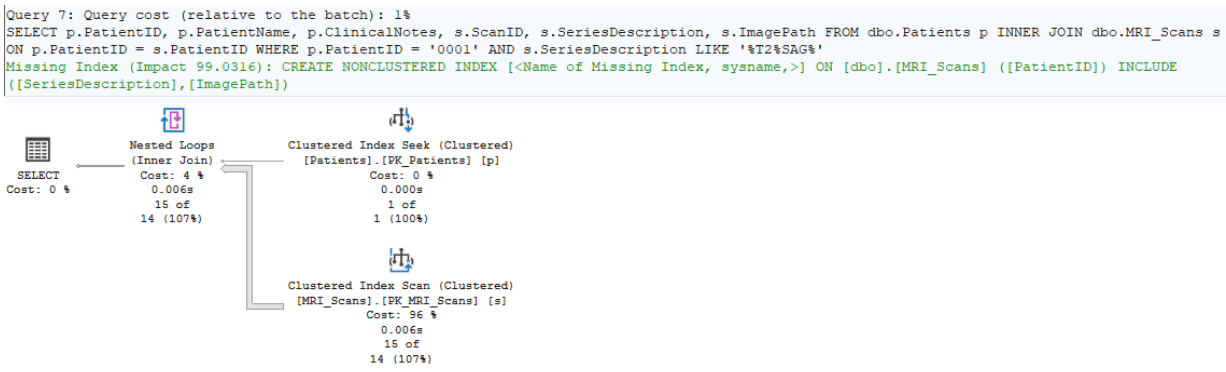


Figure 2.1: Baseline execution plan of Q1/Q2: *Clustered Index Scan* on `PK_MRI_Scans` with a Missing Index warning

Baseline (Figure 2.1). The plan displays a Missing Index warning (Impact 99.0316). The *Clustered Index Seek* on `PK_Patients` costs 0% (0.000s, 1 of 1), whereas the *Clustered Index Scan* on `PK_MRI_Scans` accounts for **96%** of the total cost (0.006s) and reads all 1,880 pages only to filter out 15 rows (Q2) or 65 rows (Q1). SQL Server itself suggests creating an index on `([PatientID]) INCLUDE ([SeriesDescription], [ImagePath])` with an estimated improvement of 99.0316%.

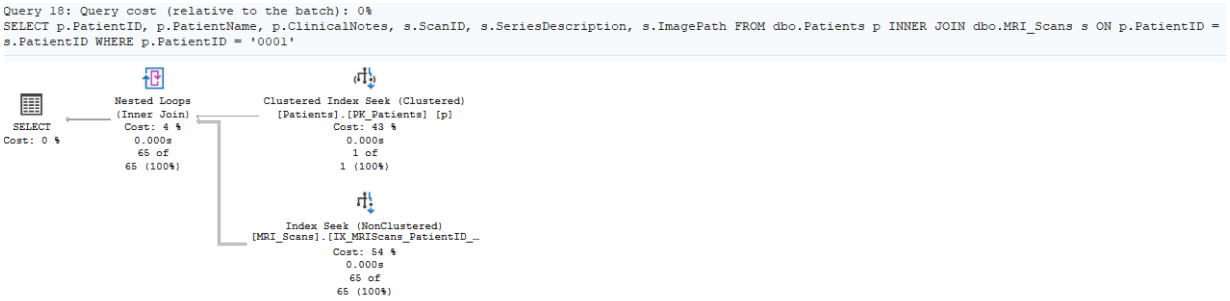


Figure 2.2: Optimized execution plan of Q1: *Index Seek (NonClustered)* on `IX_MRIScans_PatientID_Covering`

Q1 after optimization (Figure 2.2). The Missing Index warning disappears. The *Index Seek (NonClustered)* operator accounts for 54% of the relative cost, its execution time drops to 0.000s, and it retrieves exactly 65 of 65 (100%) rows. The *Nested Loops (Inner Join)* costs only 4% (0.000s). The plan contains **no Key Lookup** because `ImagePath` is included at the leaf level.

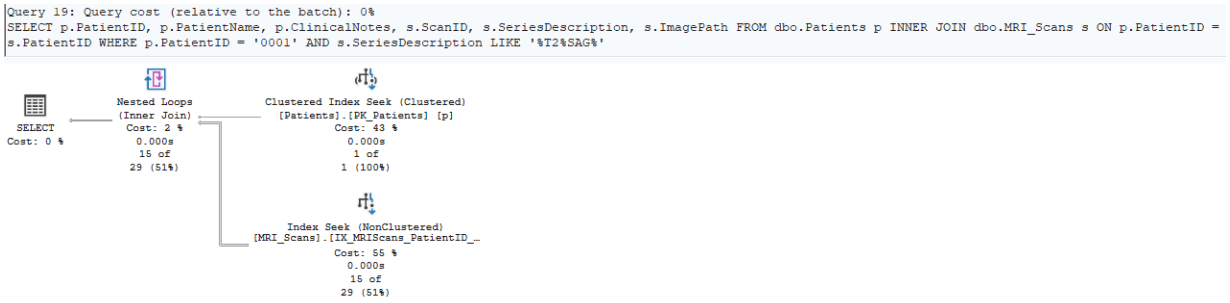


Figure 2.3: Optimized execution plan of Q2: *Index Seek* with a residual predicate

Q2 after optimization (Figure 2.3). SQL Server performs an *Index Seek* on the leading key `PatientID = '0001'` to isolate the 65 rows of the patient (reading only 6 logical pages), then applies the *residual predicate* `SeriesDescription LIKE '%T2%SAG%'` in memory, returning 15 of 29 (51%) rows in 0.000s.

2.3.2 Q3: Rowstore Parallel Scan versus Columnstore Batch Mode

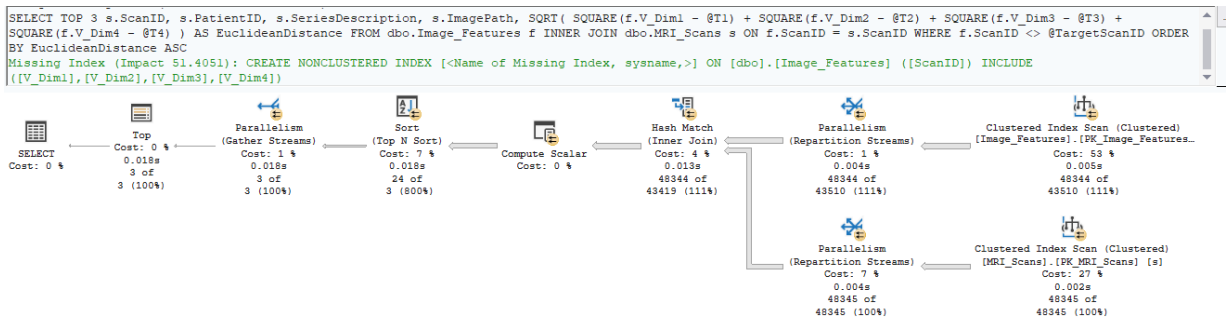


Figure 2.4: Baseline execution plan of Q3: parallel rowstore scan with three Parallelism operators

Baseline (Figure 2.4). The plan shows a Missing Index warning (Impact 51.4051) on `Image_Features` and three Parallelism operators.

- Upper right branch: a *Clustered Index Scan* on `PK_Image_Features` accounts for **53%** of the cost (0.005s), reading 3,909 pages across 9 parallel threads (Scan count 9).
- Lower right branch: a *Clustered Index Scan* on `PK_MRI_Scans` accounts for 27% of the cost (0.002s), reading 1,972 pages.
- After the *Hash Match (Inner Join)* (4%, 0.013s) and *Compute Scalar* (0%), the *Sort (Top N Sort)* operator costs 7% (0.018s) across multiple parallel threads, before *Parallelism (Gather Streams)* merges the output into the final 3 rows.

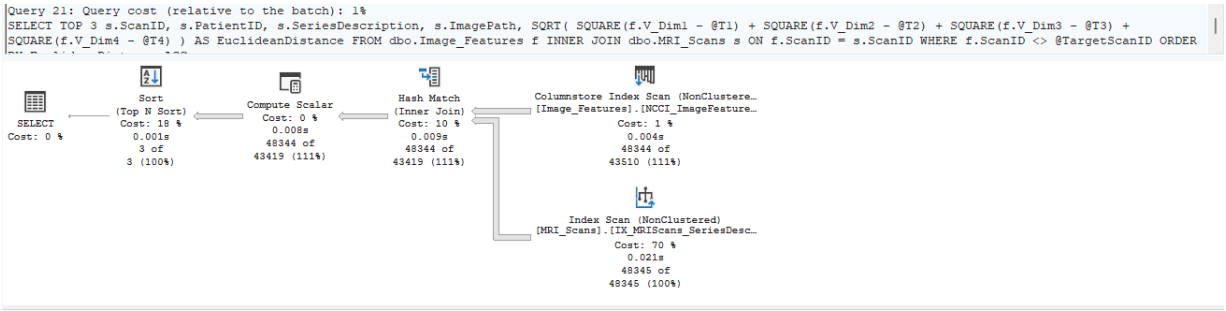


Figure 2.5: Optimized execution plan of Q3: *Columnstore Index Scan* running in Batch Mode

Q3 after optimization (Figure 2.5). The Missing Index warning disappears and the upper right branch becomes a *Columnstore Index Scan (NonClustered)* on `NCCI_ImageFeatures_Dims`.

- **`Image_Features` table:** the relative cost of the scan operator falls from 53% to only **1%** (0.004s), reading a single compressed column segment (Segment reads = 1) with 0 logical reads on the rowstore buffer pool.
- **`MRI_Scans` table:** instead of scanning the clustered index (1,972 pages), SQL Server automatically chooses the lightweight index `IX_MRIScans_SeriesDescription` (1,576 pages, a 20.1% I/O reduction).
- **Sorting:** thanks to single-threaded execution combined with Batch Mode, the *Sort (Top N Sort)* time drops from 0.018s to 0.001s (about 18 times faster) and returns 3 of 3 (100%) rows directly.

Note: the variable-assignment step `SELECT @T1 = V_Dim1 ... WHERE ScanID = @TargetScanID` (Query 20 screenshot) costs only 5 logical reads and is therefore not the focus of the analysis; the focus of Q3 is the main query that computes the Euclidean distance.

2.4 Benchmark Evaluation: I/O and CPU Measurements

Table 2.2 summarises the results measured with `SET STATISTICS IO ON` and `SET STATISTICS TIME ON` in SSMS.

Table 2.2: Benchmark results for baseline and optimized Q1, Q2, and Q3

Query	State	Reads (MRI_Scans)	Reads (Patients / Features)	Total reads	CPU (ms)	Elapsed (ms)	Main access operator
Q1	Baseline	1,880	2 (Patients)	1,882	15	72	Clustered Index Scan (96% cost)
	Optimized	6	2 (Patients)	8	0	37	Index Seek (Covering), 0 Key Lookups
	Change	*-99.68%*	*unchanged*	*-99.57%*	*-100%*	*-48.61%*	Table scan eliminated
Q2	Baseline	1,880	2 (Patients)	1,882	0	132	Clustered Index Scan (96% cost)
	Optimized	6	2 (Patients)	8	0	123	Index Seek + Residual Filter
	Change	*-99.68%*	*unchanged*	*-99.57%*	*stays <1*	*-6.82%*	Filtering on 6 leaf pages
Q3	Baseline	1,972	3,909 (Image_Features)	5,881	76	144	Parallel Clustered Index Scan (Row Mode)
	Optimized	1,576	0 + 236 LOB (Segment reads = 1)	1,812	47	163	Columnstore Index Scan (Batch Mode)
	Change	*-20.08%*	*-93.96% (incl. LOB)*	*-69.19%*	*-38.16%*	*+13.19%*	Single-threaded, Batch Mode

2.4.1 Discussion of Results

- **Q1 and Q2:** logical reads on `MRI_Scans` drop from 1,880 to 6 (**-99.68%**) and total I/O drops by 99.57%. The full table scan is completely replaced by an *Index Seek*. For Q1, CPU time falls from 15 ms to 0 ms and elapsed time drops by 48.61%; for Q2 the elapsed-time reduction is more modest (6.82%) because the remaining time does not depend on I/O against the table.
- **Q3:** total pages read fall by almost 70% (5,881 to 1,812) and CPU time falls by **38.16%** (76 ms to 47 ms). Although elapsed time rises from 144 ms to 163 ms, this is because the execution plan changes from 8 parallel threads (Scan count 9) to a single sequential thread. In exchange, the system consumes less processor time, which is advantageous when many queries run concurrently.

2.4.2 Chapter Summary

The three index structures address the specific bottleneck of each query. The Covering B-Tree Index eliminates the table scan caused by the missing foreign-key index (Q1, Q2). The Filtered Index offers a narrower index for the T2 Sagittal sequence group, working around `Msg 10735` with an explicit value list. The Non-Clustered Columnstore Index combined with Batch Mode substantially reduces I/O and CPU for the vector computation in Q3.

Chapter 3: Query Optimization and Query Processing Analysis

3.1 Introduction

This chapter examines how SQL Server processes the project's representative queries, with emphasis on query optimization, cardinality estimation, statistics, join algorithms, scalar calculations, and parallel execution. The analysis focuses on the baseline database configuration before applying the additional indexes.

The investigation uses actual execution plans, operator properties, SQL Server statistics, and runtime measurements. The execution plans provide evidence of the access paths and processing operators selected by the optimizer, while the statistics and cardinality estimates help explain its decisions. Runtime and I/O observations are used to examine the resulting execution behaviour.

3.2 Q1: Patient and MRI Scan Retrieval

3.2.1 Query Characteristics

Q1 joins `Patients` and `MRI_Scans` using `PatientID` and filters for patient `'0001'`. The query returns the patient's information together with the matching MRI scan records.

3.2.2 Execution Plan and Join Selection

Figure 3.1 presents the baseline actual execution plan for Q1. The plan uses a Clustered Index Seek on `Patients`, followed by a Clustered Index Scan on `MRI_Scans`. The two inputs are combined using a Nested Loops (Inner Join) operator.

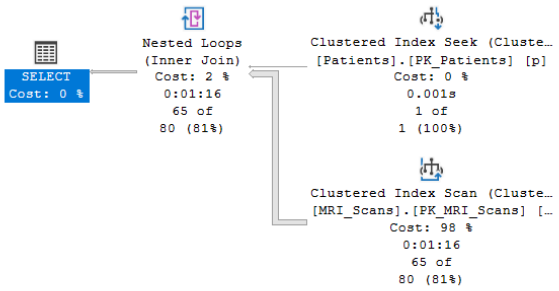


Figure 3.1: Baseline actual execution plan for Q1

The seek on `Patients` returns one row, which forms a small outer input for the Nested Loops operator. The inner input scans `MRI_Scans` because the baseline database does not have an index on its `PatientID` column. The scan reads all 48,345 rows and returns 65 matching rows.

The relevant properties of the Clustered Index Scan are summarized in Table 3.1.

Table 3.1: Q1 Clustered Index Scan properties for `MRI_Scans`

Property	Value
Actual rows read	48,345
Actual rows returned	65
Estimated rows returned	79.875
Estimated rows to be read	48,345
Actual executions	1
Estimated operator cost	1.44387 (98% of plan cost)

Figure 3.2 provides the detailed operator properties used to obtain these values.

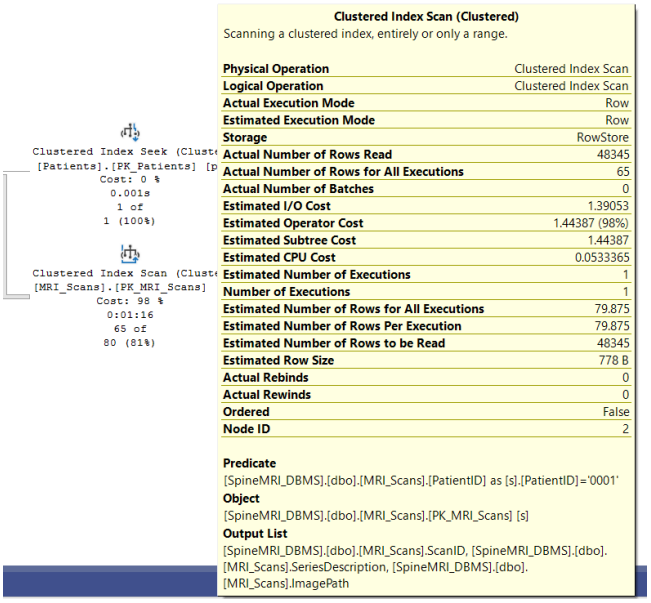


Figure 3.2: Actual and estimated properties of the Q1 Clustered Index Scan

The optimizer's choice of Nested Loops is consistent with the one-row outer input. However, the absence of an index on the inner input's join column means that the scan must examine the entire `MRI_Scans` table. Therefore, the principal inefficiency in this baseline plan is the amount of data read, rather than a large cardinality-estimation error.

3.2.3 Cardinality Estimation

The Clustered Index Seek on `Patients` estimates one row and returns one row. For the scan on `MRI_Scans`, SQL Server estimates 79.875 matching rows, while the actual number is 65.

The Q-error is calculated as:

$$Q\text{-error} = \max\left(\frac{79.875}{65}, \frac{65}{79.875}\right) \approx 1.23.$$

The estimate is therefore approximately 23% higher than the actual result. This is a relatively small estimation error for this query.

3.2.4 Statistics for `MRI_Scans.PatientID`

The following query retrieves the statistics information for `MRI_Scans.PatientID`, including its header, density vector, and histogram.

Listing 3.1: Retrieving Q1 statistics for MRI_Scans.PatientID

```
DBCC SHOW_STATISTICS (
    'dbo.MRI_Scans',
    '_WA_Sys_00000002_4BAC3F29'
);
GO
```

The statistics metadata and density information, together with histogram entries, are summarized in Tables 3.2 and 3.3, respectively.

Table 3.2: Statistics Metadata and Density Information for `MRI_Scans.PatientID`

Property	Value
Rows	48,345
Rows sampled	32,599
Sampling percentage	67.43%
Steps	200
Density	0.02050357
Average key length	4
String index	YES
Unfiltered rows	48,345
Persisted sample percent	0

Table 3.3: Histogram Entries for `MRI_Scans.PatientID`

RANGE_HI_KEY	RANGE_ROWS	EQ_ROWS	DISTINCT_RANGE_ROWS	AVG_RANGE_ROWS
0001	0	79.87501	0	1
0003	52.05838	105.0208	1	51.85342
0007	182.948	144.9583	3	60.8224
0009	74.36911	94.66667	1	74.07632
0011	69.90696	103.5417	1	69.63174
⋮				
0565	188.8975	35.5	2	94.16961
0568	80.31863	109.4583	1	80.00242
0571	239.4685	127.2083	2	119.3804
0573	65.44482	221.875	1	65.18716
0575	0.1880675	79.87501	0	95.16214

The histogram contains an exact entry for `PatientID = '0001'`, with `EQ_ROWS = 79.87501`. This corresponds to the optimizer's estimated cardinality of approximately 79.875 rows in the execution plan. Compared with the 65 actual matching rows, this represents an overestimation of approximately 23%.

This indicates that the histogram provides a reasonably close estimate for the equality predicate. However, the statistics estimate does not change the access path: without an index on `MRI_Scans.PatientID`, SQL Server still performs a Clustered Index Scan.

3.2.5 Runtime and I/O Observations

The captured execution plan reports 1,880 logical reads, 3 physical reads, and 1,874 read-ahead reads for the `MRI_Scans` scan. The query consumed approximately 369 ms of CPU time, while its elapsed time was approximately 76.3 seconds.

The substantial difference between CPU and elapsed time indicates that the query spent considerably more time waiting than actively processing data. The most prominent reported wait was `SLEEP_BPOOL_STEAL`, accounting for approximately 75.9 seconds. However, the available execution-plan data does not establish the underlying cause of this wait, so the elapsed time should not be treated as a representative measure of the scan's processing cost without further investigation.

The plan also reports that it was retrieved from cache. This means the captured execution reused a cached execution plan rather than requiring a fresh compilation. The runtime and I/O measurements are therefore recorded as observations from this particular execution, rather than as definitive performance characteristics of Q1.

3.2.6 Nested Loops versus Hash Match for Q1

To test the optimizer's join choice, Q1 was also executed with `OPTION (HASH JOIN)`. Table **??** compares the two plans.

Both plans read the same 1,880 pages of `MRI_Scans`, because the inner table must be scanned in full without an index on `PatientID`. The forced Hash Match raises the join operator cost from 0.0235 to 0.0443 and requires a

Table 3.4: Q1: Nested Loops (chosen) versus Hash Match (forced)

Metric	Nested Loops	Hash Match
Estimated subtree cost	1.47069	1.4915
Join operator cost	0.0235 (2%)	0.0443 (3%)
Memory grant (KB)	0	1,072
Degree of parallelism	1	1
Estimated / actual join rows	79.875 / 65	79.875 / 65
Logical reads, `MRI_Scans`	1,880	1,880

1,072 KB memory grant, with no spill to tempdb (zero Workfile and Worktable reads). The total estimated cost is therefore about 1.4% higher than the optimizer's chosen plan, which supports its selection of Nested Loops.

However, the scan accounts for most of the plan cost in both cases, so the join algorithm has only a minor influence on Q1. The estimated cardinality of 79.875 rows is unchanged, so the forced join does not alter the Q-error of 1.23.

3.3 Q2: Filtered Scan Retrieval by Pulse Sequence

3.3.1 Query Characteristics

Q2 retrieves the T2-weighted sagittal series of patient `'0001'` by joining `Patients` and `MRI_Scans` on `PatientID` and applying the predicate `SeriesDescription LIKE '%T2%SAG%'`. Clinically, radiologists inspect T2-weighted sagittal series to assess disc herniation and cord compression, so this filtered retrieval is a representative workload for the reporting console.

3.3.2 Execution Plan and Join Selection

Figure ?? presents the baseline actual execution plan for Q2. As with Q1, the plan combines a Clustered Index Seek on `Patients` with a full Clustered Index Scan of `MRI_Scans` through a Nested Loops (Inner Join) operator. The scan contributes 96% of the total plan cost (0.85990 out of 0.89564).

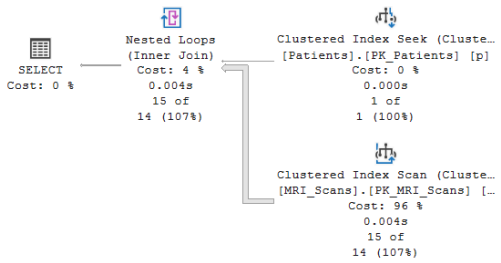


Figure 3.3: Baseline actual execution plan for Q2

The leading wildcard in the `LIKE` predicate makes it unsargable: no index can serve a filter of the form `LIKE '%T2%SAG%'`, so the predicate is evaluated as a residual filter on every row of the scan. The scan reads all 48,345 rows and returns the 15 rows of patient `'0001'` whose series description matches the pattern.

The relevant properties of the Clustered Index Scan are summarized in Table ??.

Table 3.5: Q2 Clustered Index Scan properties for `MRI_Scans`

Property	Value
Actual rows read	48,345
Actual rows returned	15
Estimated rows returned	13.8719
Estimated rows to be read	48,345
Actual executions	1
Estimated operator cost	0.85990 (96% of plan cost)

Figure ?? provides the detailed operator properties used to obtain these values.

The predicate list shown in the scan properties is `(SeriesDescription LIKE '%T2%SAG%') AND (@1 = PatientID)`. Both the `LIKE` expression and the `PatientID` equality appear as residual predicates rather

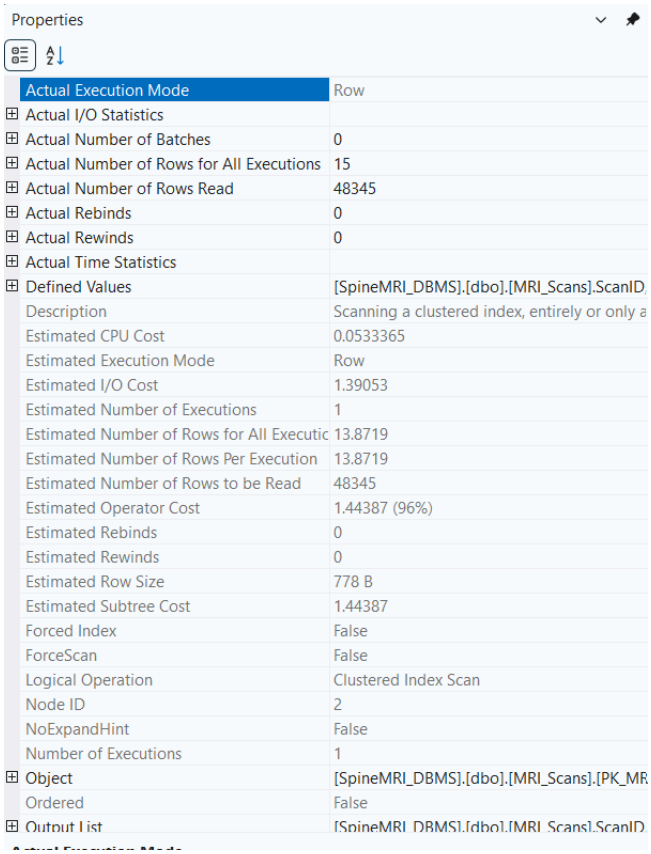


Figure 3.4: Actual and estimated properties of the Q2 Clustered Index Scan

than seek arguments, a direct consequence of the scan being the chosen access method in the absence of any index on the join column. The plan also reports a missing-index hint suggesting a nonclustered index on `MRI_Scans.PatientID` with `SeriesDescription` and `ImagePath` as included columns, with an impact of 99.0316% — the largest improvement opportunity in the plan. The optimizer therefore identifies the missing join-column index, not a `SeriesDescription` index, as the dominant optimization target.

3.3.3 Cardinality Estimation

The Clustered Index Seek on `Patients` estimates one row and returns one row. For the scan on `MRI_Scans`, SQL Server estimates 13.8719 matching rows, while the actual number is 15.

The Q-error is calculated as:

$$Q\text{-error} = \max\left(\frac{13.8719}{15}, \frac{15}{13.8719}\right) \approx 1.08.$$

The estimate is therefore approximately 8% lower than the actual result, a small error that one would not expect for an unsargable `LIKE` predicate. The reason is that the optimizer does not fall back to a flat default selectivity: dividing the estimated rows by the `PatientID` estimate (13.8719 / 79.875) gives an implied `LIKE` selectivity of approximately 0.174, which is close to the global proportion of `MRI_Scans` rows whose series description matches the pattern (9,809 out of 48,345, or 20.3%). For patient `'0001'`, all 15 matching slices belong to the single most frequent series in the database, `T2_TSE_SAG_384_0002`, so the step-level histogram estimate coincides well with the true value. For patients whose matching slices lie in the long tail of the distribution, the same estimation method would be considerably less reliable.

3.3.4 Statistics for `MRI_Scans.SeriesDescription`

The following query retrieves the statistics information for `MRI_Scans.SeriesDescription`, including its header, density vector, and histogram. The statistics object `_WA_Sys_00000004_4CA06362` is auto-created on `SeriesDescription`; auto-created statistics names are generated per database instance, so the name should be verified on other deployments (see the lookup query in `scripts/sql/03_role_3_support.sql`).

Listing 3.2: Retrieving Q2 statistics for MRI_Scans.SeriesDescription

```
DBCC SHOW_STATISTICS (
    'dbo.MRI_Scans',
    '_WA_Sys_00000004_4CA06362'
);
GO
```

The statistics metadata and density information, together with selected histogram entries, are summarized in Tables ?? and ??, respectively.

Table 3.6: Statistics Metadata and Density Information for `MRI_Scans.SeriesDescription`

Property	Value
Rows	48,345
Rows sampled	32,510
Sampling percentage	67.25%
Steps	199
Density	0.08955224
All density	0.003861004
Average key length	68
String index	YES
Unfiltered rows	48,345
Persisted sample percent	0

Table 3.7: Histogram Entries for `MRI_Scans.SeriesDescription`

RANGE_HI_KEY	RANGE_ROWS	EQ_ROWS	DISTINCT_RANGE_ROWS	AVG_RANGE_ROWS
T2_TSE_SAG_384_0002	0	7,256	0	1
T2_TSE_TRA_384_0004	5,645.2701	7,145	1	5,645.2701
T1_TSE_TRA_0005	13,404.7807	7,049	2	3,829.615
T1_TSE_SAG_320_0003	20,765.4104	6,878	3	2,115.4087
		⋮		
	last step	1	1	1

The column holds 289 distinct values across 199 histogram steps, with a heavily skewed distribution: the first step alone (`T2_TSE_SAG_384_0002`) holds 7,256 rows, and the four most frequent series codes together cover 28,328 rows, approximately 58.6% of the table. The tail of the histogram consists of many steps with `EQ_ROWS = 1`, which the optimizer samples with reduced precision.

3.3.5 Runtime and I/O Observations

The `MRI_Scans` scan performs the same 1,880 logical reads as Q1, because the scan visits the identical 1,876 leaf pages plus overhead. The difference is entirely in the result set: Q2 returns 15 rows instead of Q1's 65. The unsargable `LIKE` predicate therefore reduces the number of returned rows without reducing the I/O footprint, which confirms that any improvement must come from changing the access path (an index on the join column) rather than from the filter itself.

With a warm buffer pool, the captured run reports 0 physical reads for `MRI_Scans` and 2 for `Patients`, with 15 ms of CPU time and 27 ms of elapsed time. The Q1 capture showed the same I/O footprint with an anomalous elapsed time of approximately 76.3 seconds dominated by a `SLEEP_BPOOL_STEAL` wait, while Q2 completes in under 30 ms. The I/O behaviour of the scan is thus stable across the two queries, and the observed elapsed time is strongly dependent on buffer pool state.

As a cross-reference, the missing-index hint for Q2 matches the covering index `IX_MRIScans_PatientID_Covering` introduced by Role 2. With that index in place, the same query is served by an Index Seek that reads only the 65 rows belonging to patient `'0001'`, applying the `LIKE` predicate as a residual filter on the index rows instead of on the entire table.

3.4 Q3: Nearest-Neighbor Feature Search

3.4.1 Query Characteristics

Q3 retrieves the feature vector for target scan `ScanID = 1`, calculates the Euclidean distance between that vector and the other feature vectors, joins the feature records with MRI scan metadata, and returns the three records with the smallest distances.

The distance expression uses four feature dimensions:

$$d = \sqrt{(f_1 - T_1)^2 + (f_2 - T_2)^2 + (f_3 - T_3)^2 + (f_4 - T_4)^2},$$

where $f_1, \ldots, f_4$ are the candidate feature values and $T_1, \ldots, T_4$ are the target feature values.

3.4.2 Execution Plan and Join Algorithm

Figure 3.3 presents the baseline actual execution plan for the main nearest-neighbor query.

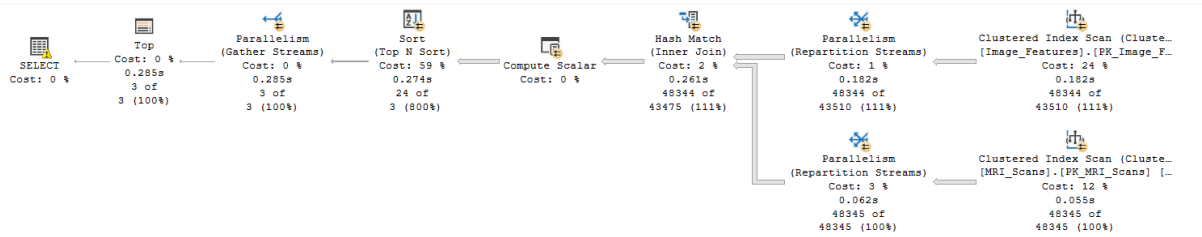


Figure 3.5: Baseline actual execution plan for Q3

Unlike Q1 and Q2, the baseline plan uses a Hash Match (Inner Join) to combine `Image_Features` and `MRI_Scans`. Both tables are accessed through Clustered Index Scans. The results are repartitioned by `ScanID` before being processed by the parallel Hash Match operator.

The Hash Match operator estimates 43,475.4 output rows and produces 48,344 actual rows. Its Q-error is:

$$Q\text{-error} = \max\left(\frac{48,344}{43,475.4}, \frac{43,475.4}{48,344}\right) \approx 1.11.$$

The optimizer therefore underestimates the join output by approximately 10.1%. Compared with Q1 and Q2, Q3 joins substantially larger inputs, making Hash Match an appropriate join strategy for this workload.

The relevant scan and join properties are summarized in Table 3.4.

Table 3.8: Q3 scan and join cardinality

Operator	Estimated rows	Actual rows
`Image_Features` scan	–	48,345 read; 48,344 returned
`MRI_Scans` scan	–	48,345 read; 48,345 returned
Hash Match	43,475.4	48,344

The `Image_Features` scan applies `ScanID <> @TargetScanID` as a residual predicate, returning 48,344 rows. The `MRI_Scans` scan reads all 48,345 rows.

3.4.3 Parallel Execution

The main Q3 query uses a degree of parallelism of eight. Figure 3.4 highlights the parallelism operators and data flow surrounding the Hash Match operator.

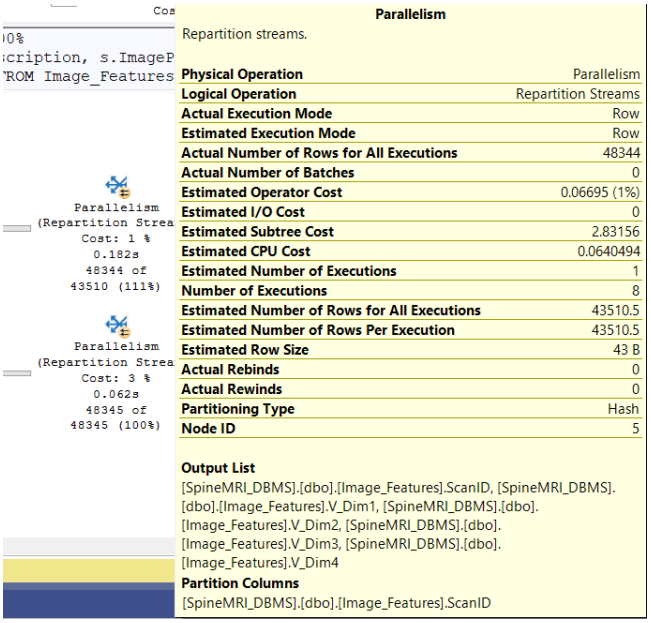


Figure 3.6: Parallel data flow in the Q3 execution plan

The plan uses Repartition Streams to distribute rows by `ScanID` before the join and Gather Streams to collect the sorted results. The per-thread counters for the Hash Match operator sum to 48,344 rows across the eight active workers.

The Top-N Sort performs local sorting with `Top Rows = 3` on each worker. This produces up to 24 local candidates before the results are gathered. A final Top operator then returns the three globally closest records.

The separate statement that retrieves the target feature vector has a degree of parallelism of one. Therefore, the parallel execution described here applies to the main nearest-neighbor query rather than the preliminary lookup.

3.4.4 Statistics Used for Cardinality Estimation

The following query retrieves the statistics information for `Image_Features.ScanID`, including its header, density vector, and histogram.

Listing 3.3: Retrieving Q3 statistics for Image_Features.ScanID

```
DBCC SHOW_STATISTICS (
    'dbo.Image_Features',
    'UQ__Image_Fe__63B32660DF05EFC2'
);
GO
```

The header of the statistics object `UQ__Image_Fe__63B32660DF05EFC2`, associated with `Image_Features.ScanID`, is summarized in Table 3.5.

Table 3.9: Statistics Metadata and Density Information for `Image_Features.ScanID`

Property	Value
Rows	48,345
Rows sampled	18,900
Sampling percentage	39.09%
Steps	175
Density	2.377895E-05
Average key length	8 bytes
String index	NO
Unfiltered rows	48,345
Persisted sample percent	0

The histogram for `ScanID` contains 175 steps, with selected entries summarized in Table 3.6. Each step describes the distribution of values up to its corresponding range boundary.

Table 3.10: Histogram Entries for `Image_Features.ScanID`

RANGE_HI_KEY	RANGE_ROWS	EQ_ROWS	DISTINCT_RANGE_ROWS	AVG_RANGE_ROWS
40	0	1	0	1
265	241.8147	1	224	1.07953
655	264.9672	1	265	1.001525
1070	162.0673	1	162	1
1445	326.7071	1	326	1.002637
⋮				
47322	326.7071	1	205	1.593693
47580	326.7071	1	257	1.271234
48020	326.7071	1	326	1.002637
48296	272.6846	1	272	1.001691
48297	0	1	0	1

Although these statistics describe the indexed join column, the histogram alone does not explain the optimizer's complete cardinality estimate for the Q3 join. The execution plan also uses statistics from the other participating tables and columns. Therefore, the individual contribution of each statistics object to the estimated join cardinality cannot be determined from the available evidence.

3.4.5 Compute Scalar and Vector Distance Calculation

The Compute Scalar operator defines the expression `Expr1002`, which evaluates the four-dimensional Euclidean distance. The expression performs subtraction and squaring for each dimension, adds the four squared differences, and applies the square-root function. The resulting scalar value is used as the sort key for ranking candidate scans.

Figure 3.5 presents the Compute Scalar operator and its expression properties.

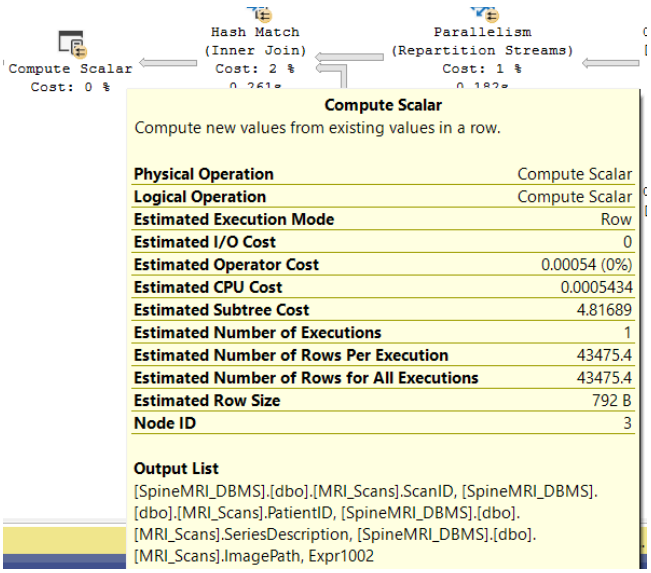


Figure 3.7: Compute Scalar expression for the Euclidean distance

The inspected plan exposes the defined expression and an estimated row count of 43,475.4. In the latest execution-plan capture, the Compute Scalar operator does not expose separate actual-row or execution counters. Consequently, no standalone actual-row count or runtime should be inferred from its properties.

The expression is evaluated as part of the query's processing pipeline before the Top-N Sort uses the calculated distances to select candidate records. This makes the scalar calculation an important part of Q3's workload, although its individual CPU cost cannot be isolated from the available operator properties.

3.4.6 Top-N Sort and Result Selection

The plan contains a Top-N Sort ordered by `Expr1002` in ascending order, with `Top Rows = 3`. Figure 3.6 shows the sorting operator and its relevant properties.

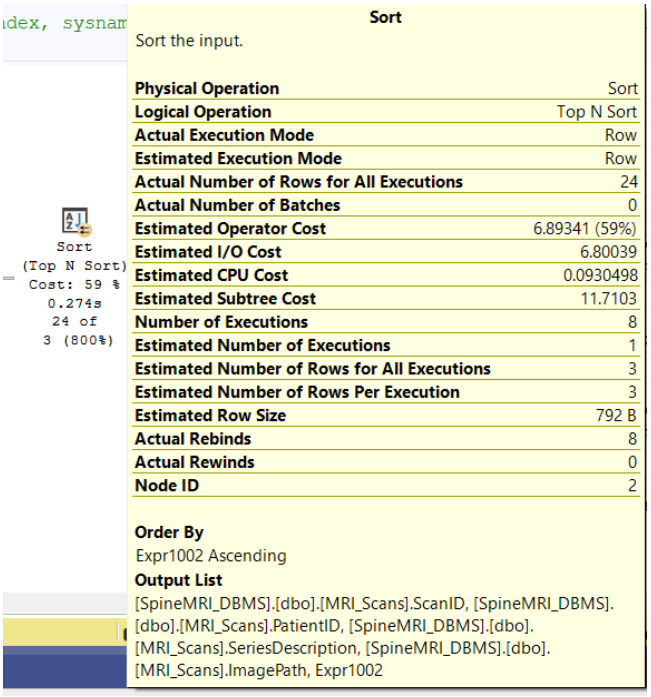


Figure 3.8: Top-N Sort used to rank candidate scans by Euclidean distance

The parallel workers perform local Top-N sorting, retaining up to three candidates each. Gather Streams combines these local results, after which the final Top operator returns the three records with the smallest global distances.

This approach avoids sorting every candidate into a complete ordered result set. Instead, the execution plan retains a limited number of candidates from each worker before producing the final result.

3.4.7 Reference-Vector Lookup

Before executing the main nearest-neighbor query, SQL Server retrieves the four feature values for the target scan. This statement uses a Nested Loops plan consisting of an Index Seek on the unique nonclustered index associated with `Image_Features.ScanID`, followed by a Clustered Index Seek on `PK_Image_Features` using the retrieved `FeatureID`.

Figure 3.7 shows the execution plan for this preliminary lookup.

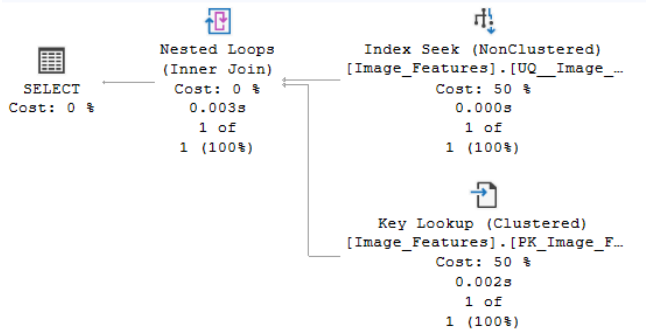


Figure 3.9: Execution plan for retrieving the target feature vector

Both seek operations return one row. The statement completes in approximately 3 milliseconds, with approximately 3 milliseconds of CPU time. Its execution is therefore negligible compared with the main nearest-neighbor query.

3.4.8 Runtime, I/O, and Memory Observations

The latest execution-plan capture reports an elapsed time of approximately 294 milliseconds and a CPU time of 751 milliseconds for the main query. CPU time exceeds elapsed time because the query executes in parallel, with CPU time accumulated across multiple worker threads.

The two Clustered Index Scans each read 48,345 rows. Their aggregate logical reads, calculated by summing the per-thread counters, are approximately 3,909 for `Image_Features` and 1,972 for `MRI_Scans`. The coordinator thread reports two physical reads for each table, while the worker-level physical reads are zero. The corresponding read-ahead reads are approximately 3,720 for `Image_Features` and 1,874 for `MRI_Scans`.

The main query receives a memory grant of 27,648 KB and reports a maximum memory usage of 8,128 KB. The plan also records a memory-grant wait of approximately 106 milliseconds. Thus, the granted memory exceeded the maximum amount used during this execution.

The wait statistics report `LATCH_EX` as the largest wait, at approximately 1,517 milliseconds, followed by `CXSYNC_PORT` at 327 milliseconds and `CXPACKET` at 168 milliseconds. These wait times are accumulated across execution threads and can therefore exceed the query's elapsed time. The execution plan does not establish the specific cause of the reported latch waits.

The latest capture also differs from the earlier observation, which reported approximately 78 milliseconds of elapsed time and 303 milliseconds of CPU time. Although both captures reused cached execution plans, their runtime measurements differ. The latest capture is used here as the current runtime observation; the difference indicates that execution time can vary between runs and should be evaluated using repeated measurements under consistent conditions.

3.4.9 Hash Match versus Nested Loops for Q3

The baseline Q3 execution plan uses a parallel Hash Match (Inner Join) with a degree of parallelism of 8. To examine the optimizer's join algorithm selection, an alternative plan was generated using `OPTION (LOOP JOIN)`. Table **??** compares their estimated execution plan properties.

Table 3.11: Q3: Hash Match versus Nested Loops

Metric	Hash Match	Nested Loops
Estimated subtree cost	11.7389	22.4434
Join operator cost	0.21353 (2%)	0.02569 (0%)
Sort operator cost	–	6.8935 (31%)
Memory grant (KB)	27,648	1,024
Degree of parallelism	8	8
Estimated rows	43,475.4	43,475.4

The Hash Match plan has an estimated subtree cost of 11.7389, compared with 22.4434 for the forced Nested Loops plan. The Nested Loops alternative therefore has approximately 91.2% higher estimated cost.

Although the Nested Loops operator has a lower estimated cost of 0.02569 than the Hash Match operator at 0.21353, its complete execution plan is more expensive which contains a Sort operator with an estimated cost of 6.8935, accounting for 31% of the plan cost.

Both plans estimate 43,475.4 output rows. The baseline Hash Match plan produces 48,344 actual rows, resulting in a Q-error of approximately 1.112. The join algorithm change therefore does not alter the estimated cardinality.

Overall, the comparison supports the optimizer's choice of Hash Match because it has the lower estimated subtree cost. The Nested Loops plan's additional sorting requirement is an important contributor to its higher estimated cost.